

1. How did you ingest and structure the data from the website?

- I used requests and **BeautifulSoup** to fetch data from Amenify pages , stripped noisy HTML(Scripts , footers etc) and normalized the text. [scraper.py]
- The clean text is then saved in [knowledge_base.json].
- [ingestion.py] slices text into chunks(500 chars) using **RecursiveCharacterTextSplitter**.
- Chunks are embedded using **gemini-embedding-001** and stored in **ChromaDB**(Vector Database).

2. How did you reduce hallucinations?

Three layers:

- **Structural grounding:** Added a system prompt — "You must ONLY answer questions using the provided context". The model cannot rely on its parametric memory.
- **Explicit fallback instruction:** If the answer is not in the context, the model is strictly instructed to reply with a hardcoded "I don't know" message plus the support email/phone.
- **Low temperature (0.0):** Using **gemini-2.5-flash-lite** with a 0.0 temperature eliminates stochastic variation and creative guessing.

3. What are the limitations of your approach?

- **Local Vector Database:** ChromaDB stores vectors in a local SQLite file. In a stateless container environment like Cloud Run, this either requires re-indexing on cold starts or complex volume mounts.
- **Static knowledge base:** Content is only as fresh as the last **scraper.py** run. New services won't be reflected until a manual re-scrape.
- **In-memory sessions:** Chat history is stored in a Python dictionary. It will be lost on container restarts and won't work in multi-instance auto-scaling deployments.
- **No authentication:** Any caller can hit the API and consume your Google API rate-limit quota.

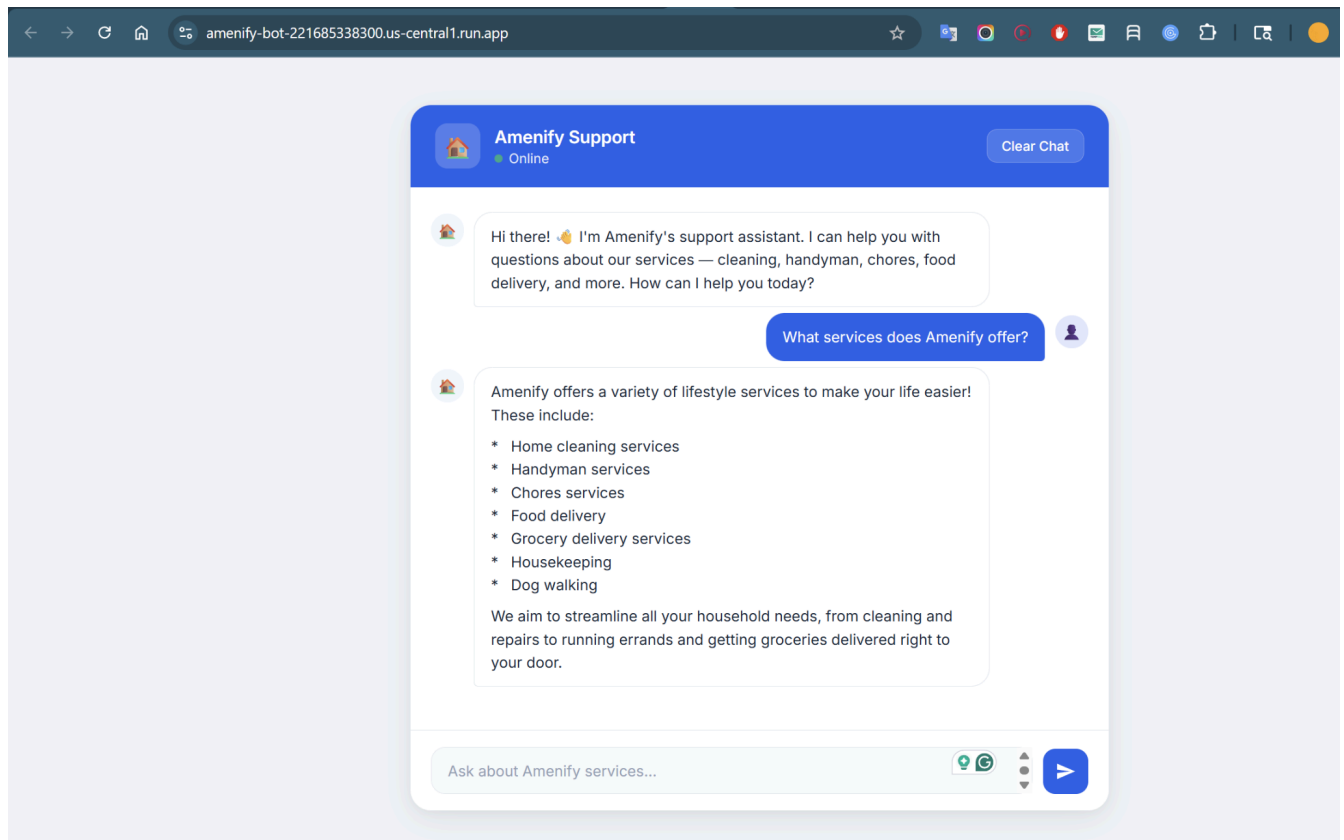
4. How would you scale this system?

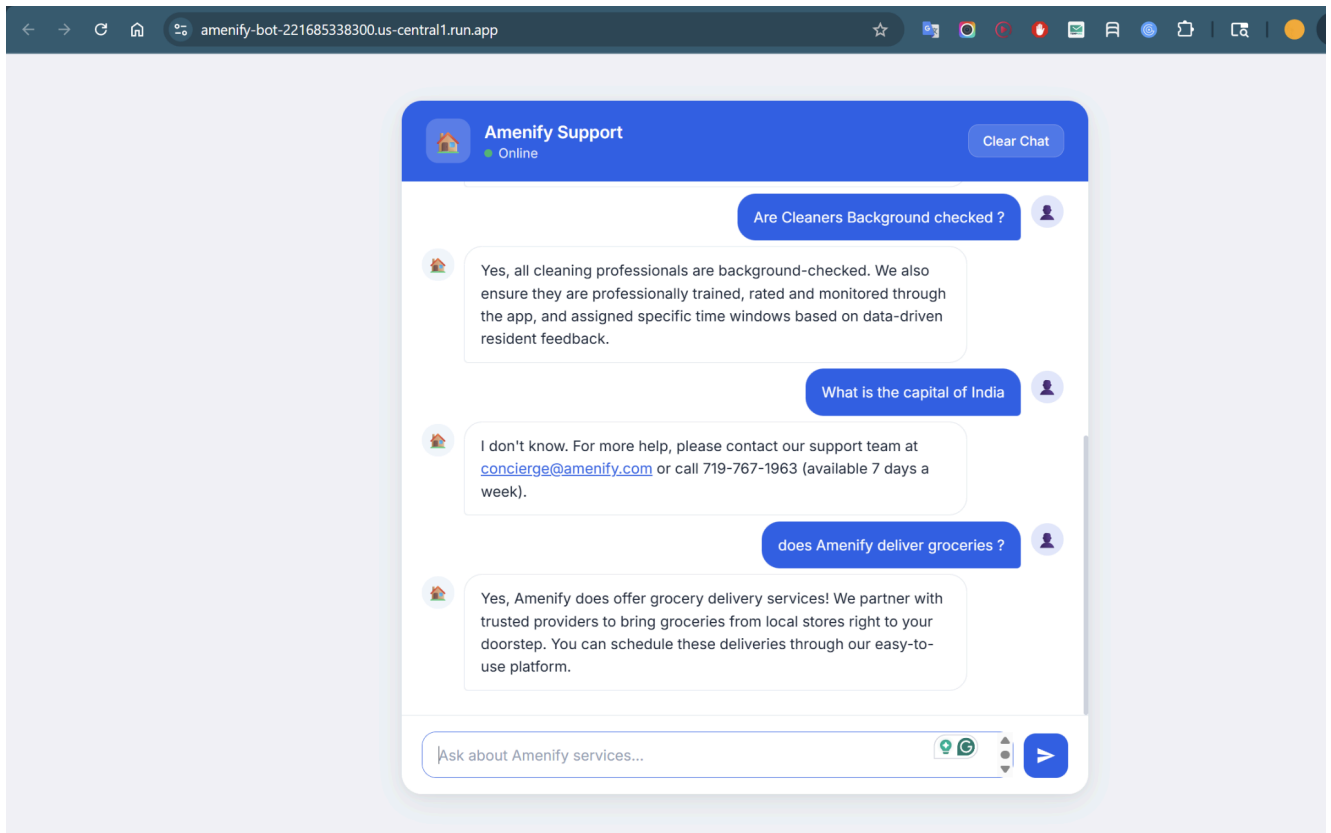
- Move vector storage from local ChromaDB to a **managed vector DB** (e.g., Google Vertex AI Vector Search or Pinecone) to decouple storage from compute.
- Move to Distributed Sessions (**Redis**) with a TTL so horizontal load balancing across multiple Cloud Run instances works seamlessly , to absorb infinite traffic while retaining user context.
- Add a **rate limiter** (e.g., via FastAPI middleware) to prevent quota abuse.
- Schedule **nightly scrape jobs** (Cloud Scheduler + Cloud Run Jobs) to automatically keep the knowledge base up to date.
- Put the API behind a **CDN/WAF** (Cloudflare or Google Cloud Armor) for DDoS protection.

5. What improvements would you make for production use?

- **Retrieval:** I would upgrade ChromaDB(Similarity Search) to use Hybrid search: combine dense vector embeddings with BM25 keyword search, followed by cross-encoder re-ranking.
- **Auth:** JWT/API-key auth on /chat; origin validation and per-key rate limits.
- **Automated Knowledge syncing:** Scheduled Cron-job to periodically crawl Amenify website, and automatically update the VectorDB.
- **Telemetry and Auditing:** Integrate tracing tool(eg, LangSmith) to monitor token ingestion costs, user drop-offs, log every query, context, reply for analysis.
 - Track and aggregate "I don't know" responses to actively identify missing gaps in the website documentation.

6.Example queries and outputs





Prior Experience with Production RAG Architectures

Architecting this solution aligns directly with production systems I regularly build. Here is how I have tackled similar RAG and pipeline challenges in my recent projects:

- **Minimizing Hallucinations in High-Stakes Domains:** I engineered **JurisOne**, an AI Legal Assistant utilizing LangChain and Pinecone to retrieve complex case laws. I applied strict grounding techniques to ensure factual accuracy over dense legal text, drastically reducing LLM hallucinations.
- **Scalable Retrieval & FastAPI Integration:** For my **Agentic AI Portfolio Manager**, I deployed a production-ready ChromaDB pipeline wrapped in FastAPI to query SEBI compliance documents and execute real-time data tracking.
- **Automated Data Freshness (CI/CD):** To solve the "static knowledge base" limitation present in this Amenify bot, I regularly implement auto-ingestion pipelines using **GitHub Actions**. By setting up CI/CD workflows, the system automatically triggers web scraping, chunking, and vector database re-indexing whenever source content updates, ensuring the LLM context is never stale without requiring manual intervention.